Learning-Based Trajectory Optimization for UAVs

M. Suphi Deniz Karanfil

Individual Project, Winter 2023-2024

Advisor: Robert Penicka

1. Introduction

Unmanned Aerial Vehicles have gained significant attention and application in various fields including surveillance, transportation, delivery services, and communication platforms. The ability of UAVs to efficiently navigate in the selected environments stems from two main parts which are route planning and trajectory generation. While route planning provides a trajectory skeleton consists of waypoints to follow under predefined constraints time maximum flight time, trajectory generation deals with detailed planning of the UAV's motion between successive waypoints with considerations to dynamic boundaries such as max acceleration for the vehicle [1]. Because there a lot of dynamic parameters to consider, time optimality of trajectory generation algorithms is also crucial to UAV's performance. Within this work, true trajectory data has been generated by previous work by Teissing [2], in which gradient-method-based algorithm was incorporated with an iterative process for better estimations of trajectory acceleration distribution to find the best available trajectory. The related data, then, has been used to train and evaluate a deep neural network created by default libraries like Torch with the hope that it can return sufficient estimations in much shorter time and the evaluation process of the DNN has demonstrated its rapid and accurate estimations.

The problem formulations for route planning and trajectory generation exhibit distinct objectives and considerations. Route planning primarily focuses on determining an optimal path from a starting point to a destination which can be defined as a Traveling Salesman Problem (TSP) or an Orienteering Problem (OP), maximization of collected prizes defined by heuristics under maximum flight time. On the other hand, trajectory generation involves generating a sequence of waypoints and control inputs to guide the UAV along a specific path and trajectory optimization involves finding the most efficient path for a UAV to follow while respecting physical constraints such as maximum velocity and acceleration to accomplish its mission objectives. Both these problems are considered to be NP-hard, meaning finding exact solutions to these problems may require exponential time as the size of the input increases.

Several approaches have been proposed in the literature to address the issue of time-optimal trajectory generation for UAVs. One approach is the use of Kinematic Orienteering Problem combined with time-optimal trajectory generation [2]. The Kinematic Orienteering Problem is a problem formulation that aims to prioritize and select the most important locations for a UAV to visit within a given time frame, taking into account its kinematic properties such as velocity and acceleration limits. This approach allows for the generation of time-optimal trajectories that satisfy the physical capabilities of the UAV. Another approach to trajectory optimization for UAVs is learning-based methods. Learning-based trajectory optimization for UAVs involves using machine learning algorithms to learn and optimize the UAV's trajectory based on a set of predefined objectives and constraints. These algorithms utilize techniques such as reinforcement learning, deep learning, and genetic algorithms to train the UAVs to make informed decisions and adapt their trajectories in real time based on environmental factors and other relevant parameters. Learning-based trajectory optimization for UAVs offers several advantages such as easy adaptability for UAVs to their trajectories based on real-time information, elimination the need for manual design and parameter tuning, as the algorithms can automatically learn and adjust based on training data.

As proposed in this work, a DNN can be utilized with techniques of supervised learning to train the algorithm and make informed decisions about trajectory planning. The proposed DNN has

firstly been developed for a simple TSP, in which the DNN takes 2-d waypoints for path and series of locations to choose from as input and outputs the additional length of a path that would cause from inserting a new location to the path among various locations, enabling the selection that would result with minimum time. Secondly, it has been modified and trained with datasets of trajectories generated by gradient-based velocity optimization using per axis with a point-mass model, in which the DNN takes inputs of 3-d start position, start velocity, goal position, goal velocity, gate position to pass between start and goal position and outputs the velocity vector to pass at the gate and total time it would take the finish the path. Thus, the DNN can further be utilized to plan an entire trajectory by iteratively evaluating each of the path segment until it reaches its destination.

2. Preliminary Material and Related Work

Before proceeding to further details, some term's definitions will be stated as they can be found in the relevant literature. Path planning in three-dimensional (3D) space as in this study involves defining a workspace with obstacles, where the goal is to find a path from an initial point to a goal region while avoiding obstacles. While path planning is not a part of this study, it is a related problem and cannot be avoided when the subject is trajectory generation. Path planing can be defined as given a function of bounded variation like start and goal positions, if there exists a process that guarantees the UAV remains within the free workspace (without obstacles) for all time intervals, it is considered path planning. If a process finds a path with the minimum cost among all feasible paths, it is considered the optimal path, and the process is optimal path planning.

Following the completion of path planning, the subsequent step involves trajectory generation to determine the specific movement and control inputs necessary for the unmanned aerial vehicle (UAV) to navigate along the planned path. Trajectory planning involves determining how to move along a path, whether continuous or discrete, connecting the start and end nodes. Trajectories, often parameterized by time, are described mathematically as smooth polynomial functions, allowing computation of velocities and accelerations [5]. Trajectory planning aims to find smooth and continuous trajectory segments to follow the path effectively while respecting the dynamic constraints of the UAV. If there should be an optimality parameter such as time, the returned trajectory can further be optimized to be denoted as optimal trajectory.

The studies about UAV's path planning and optimal trajectories has shown quite a rise as the application of areas of UAV's have also increased in variety. Moreover, a subproblem of path planning namely Vehicle Routing Problem (VRP) has been a commonly researched problem in transportation and operation areas [4]. In the study of [4], authors have put the solution methods of optimization algorithms for VRP into three categories: exact, heuristic, and Learning-Based Optimization (LBO) algorithms and it was stated that while exact algorithms offer optimality, they struggle with scalability, and heuristic algorithms lack guaranteed optimality and have high computational complexity. LBO algorithms, however, have gained attention due to their ability to learn optimization strategies from training data, allowing for rapid solution generation for online tasks. LBO approaches can be classified as end-to-end or step-by-step, both of which show promising results but face challenges like data limitations and generality issues.

Despite the popularity of LBO algorithms in solving VRP, there's a lack of comprehensive assessment experiments and in-depth analyses on their characteristics. Therefore, the paper aims to fill this gap by providing an overview of LBO applications in VRP, discussing the advantages and disadvantages of different LBO algorithms through extensive experiments, and suggesting future

research trends. The contributions of the paper include aiding beginners in understanding the field's development, providing insights from experimental comparisons, and proposing promising research directions

In the article [2] authors delves into how the Orienteering Problem can be applied to the trajectory optimization of multirotor UAVs, given the constraint of maximum flight time due to limited battery capacity. The OP is a way to prioritize and select the most important locations for data collection within a given time frame. This paper enhances this approach by incorporating full kinematic properties of UAVs into the trajectory optimization process, specifically accounting for bounds on velocity and acceleration. The Kinematic Orienteering Problem as defined in this work, is an advancement over conventional approaches such as Dubins paths, which assume constant velocity and do not fully utilize the agility of multirotor UAVs, especially in sharp turns. Unlike Bézier curves, which allow for acceleration but lack intrinsic considerations for UAV physical limitations and do not necessarily provide time-optimality, the KOP ensures that the trajectories are both physically feasible and time-optimal. An exact mixed-integer formulation and a heuristic method called Large Neighborhood Search were proposed to solve the KOP. The solutions obtained through these methods were compared with those from the well-established Dubins Orienteering Problem, demonstrating the new approach's effectiveness. Moreover, the paper shows that the resulting trajectories can be precisely tracked by modern UAV flight controllers, which are based on Model Predictive Control. The paper also identifies a gap in the literature by pointing out that current methods for generating time-optimal trajectories in multiple dimensions are not generally correct. To address this, it presents an improved analytical method for trajectory generation that accounts for multi-dimensional aspects. The study concludes by envisioning enhancements to the computational efficiency and quality of heuristic solutions for the KOP and exploring the KOP in three-dimensional scenarios.

The study [3] is not only a reference for writing part of this project but also a kep component of the provided solution in this project. Therefore the solution methods described there are explained in the section 4.1.

3. Pre-Study with TSP and Getting Familiar with Used Modules

Before proceeding to main work, a simple TSP was solved by using default libraries like Torch to create the NN and Optuna to optimize hyperparameters of the NN. In this problem, positions were defined in 2 dimensional space and the NN was only for estimating the total distance after an addition of new waypoint between the start and gate position, given a list of waypoints to pass before reaching the goal, UAV’s mission to visit one more location to gather more prize such as data collection while minimizing the total distance, therefore energy consumption.

For this purpose, torch and Optuna were combined along with other libraries to create such a network, leveraging their respective strengths in deep learning framework and hyperparameter optimization. Torch, a popular deep learning framework, provided the foundation for building and training the neural network model. Its flexibility and efficiency in implementing complex neural architectures allowed for the creation of a custom model tailored to the specific problem of estimating total distance in the context of the Traveling Salesman Problem (TSP). Optuna, on the other hand, offered a powerful hyperparameter optimization framework, enabling the automatic

search for the optimal set of hyperparameters for the neural network model. By leveraging Optuna's algorithms for efficient hyperparameter search, the network could be fine-tuned to achieve better performance in terms of estimating the total distance accurately and efficiently.

Combining torch and Optuna facilitated the development of a robust neural network model for the preliminary study. The integration of these tools allowed for the creation of a tailored solution that optimized both the architecture of the neural network and its hyperparameters. However, the job can still be challenging because it needs troubleshooting until good estimations can be achieved. This process often requires meticulous attention to detail and a willingness to experiment with various strategies. For instance, ensuring tensors are not unnecessarily copied to maintain auto-tracking of gradients is crucial for efficient training. Additionally, verifying the initialization of layer parameters can significantly impact the model's performance. Exploring different optimizer and loss function options can also yield improvements in training dynamics and convergence. Leveraging Torch's dataset and dataloader functionalities proves essential in ensuring that input data is properly preprocessed, shuffled, and batched, thereby enhancing the overall robustness of the neural network model.

The model created to estimate length of a path was trained with randomly generated data from uniform distribution, where true distances were calculated using norm function in torch, given in equation 01. One should be careful while using any function in the training loop of a torch model because whenever there should be an operation that is not differentiable, the auto-track of gradient calculation is lost and those operations even include numpy based operations since the data is detached from a tensor and thus cannot be tracked after the detachment.

$$distance = \|gate_{x,y} - start_{x,y}\| + \|goal_{x,y} - gate_{x,y}\|$$

Equation 01

For the training 18000 sample of data points were generated and although in the training loss function was also defined as L2 norm of the differences to keep track of the gradients the model was evaluated by using percentage based error, as given in equation 2.

$$\%Error = \frac{target - output}{target} * 100$$

Equation 02

After considerable amount of trials with the help of optuna library, the model which returned the minimum error was selected to be the best and related loss values of the model is given in the Figure 01.

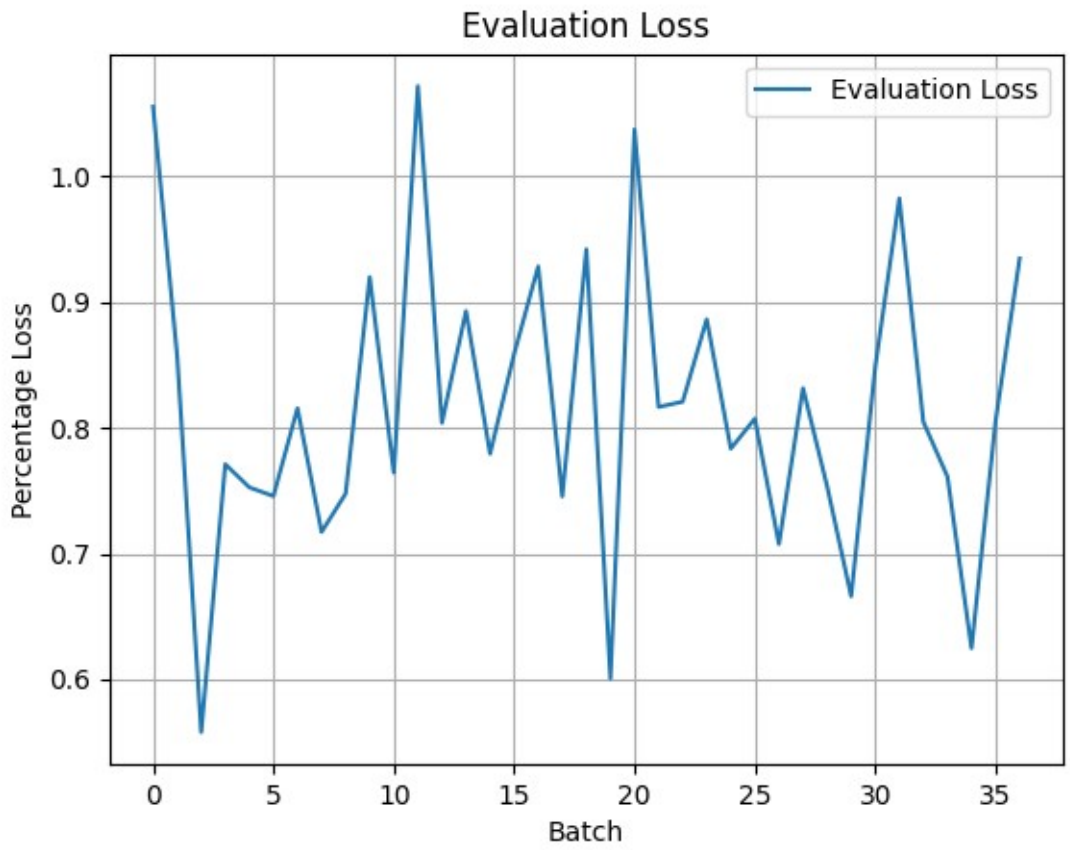

Figure 01

4. Main Study

In this main study, the problem formulation was extended to incorporate velocities, thus transforming it into a Kinematic Orienteering Problem (KOP). Additionally, spatial dimensionality was expanded to to three dimensions (3D). Therefore the model's hidden layers and sizes were also increased to capture the underlying relations between the inputs and the target outputs. Further advancements include the dataset creation, which was made possible by the works of [3]. Additionally, various neural network optimization techniques were employed to enhance the model's performance. Techniques such as different optimizer options, or initialization techniques were utilized to fine-tune the model's architecture and improve its ability to capture the underlying relations between the inputs and the target outputs. These efforts were essential in ensuring the effectiveness and reliability of the proposed approach.

4.1. Dataset Generation

A previous cpp project by [3] was the main component of the generated dataset. The mentioned project was providing solutions for each possible combinations of related positions and velocities, namely start, goal and gate. The solutions were for generating and re-planning collision-free trajectories for multirotor UAVs, especially in applications where quick, real-time responses are necessary, such as search and rescue operations. The solution method can be summarized step-by-step as following: the study proposes the use of a gradient-method-based algorithm to compute and

optimize approximate trajectories. This method relies on a simplified point-mass model with limited acceleration, allowing for quick, real-time computations. Then it includes an iterative process for the distribution of trajectory acceleration to improve the accuracy and reflect the constraints of multirotor UAVs more precisely. This iteration accounts for the effects of gravity to optimize the flight path. After that it uses methods of path planning to guide the trajectory planning process, which utilize paths with various topologies to enhance the adaptability and efficiency of trajectory generation. The overarching goal of these methods is to enable UAVs to adapt their flight paths dynamically and efficiently in complex environments, ensuring safety, reliability, and swiftness of operations. The solutions were collected with randomly generated input points drawn from uniform distribution. However, it should be noted that the selection of gate positions were bounded with those of start and goal, there were no feasible paths otherwise.

In summary, a previous project was modified in order to capture the true values for both velocity vector at the gate and total time for the UAV to finish the path with given inputs. The whole data was about 180000 samples then was saved for training process.

4.2. DNN Structure

The DNN has been created with the help of optuna library. The main parameters such as number of layers, batch size, learning rate have been selected randomly within given range and then the model with the best evaluation result have been returned by hyperparameter optimization study. Then the ranges have always decreased to the intervals where the next best model can be searched and by doing so the model has been made only more accurate. The hyperparameters' importance to the finalized NN structure is given in Figure 02, where red line is the velocity vector and the blue is time output.

Figure 02

One main concern that couldn't be handled with Optuna was that the model had two different outputs: a velocity vector at the gate and total time, which made everything more complicated. Because the ranges of the outputs had also big discrepancy whenever a backward operation is done, the gradients were accumulating for both of them at the same time. This means that each of parameters were updated based on both of them and optimizer was not able to return valid estimations for both of the outputs. While one of them was getting more accurate the other one was going worse. To overcome this, firstly two different optimizers were initialized with two

different models to keep track of each of the model's parameters separately. The velocity vector was given as an output from the first model, then with taking that as an input was given to another model with initial inputs to deliver time as an output but this was not an option either because whenever a loss function is made backwards, all of the gradients until the first input are computed once again. So, different initialization techniques were tried for the linear layer's initialization by following the guides in the torch library and it was seen that the best way to initialize the linear layers with the task in hand, was kaiming initialization, where the samples were picked from a normal distribution centered at zero and with variance of $2/sizeoftheparameter$.

4.3. Evaluation of Best Models

The models were evaluated within each trial based on the percentage error of differences. The resulting network model is demonstrated in the figure below.

4.4. Results

After comprehensive evaluation of the models, the best model was selected and outputs of the model were visualized for better understanding. Velocity vectors were selected to be shown in 3d space to make comparison easier as it can be seen in Figure 04, where green vectors represent the target vectors, red ones are the outputs given by the model. On the other hand, time outputs were just calculated as percentage error and given as a graph.

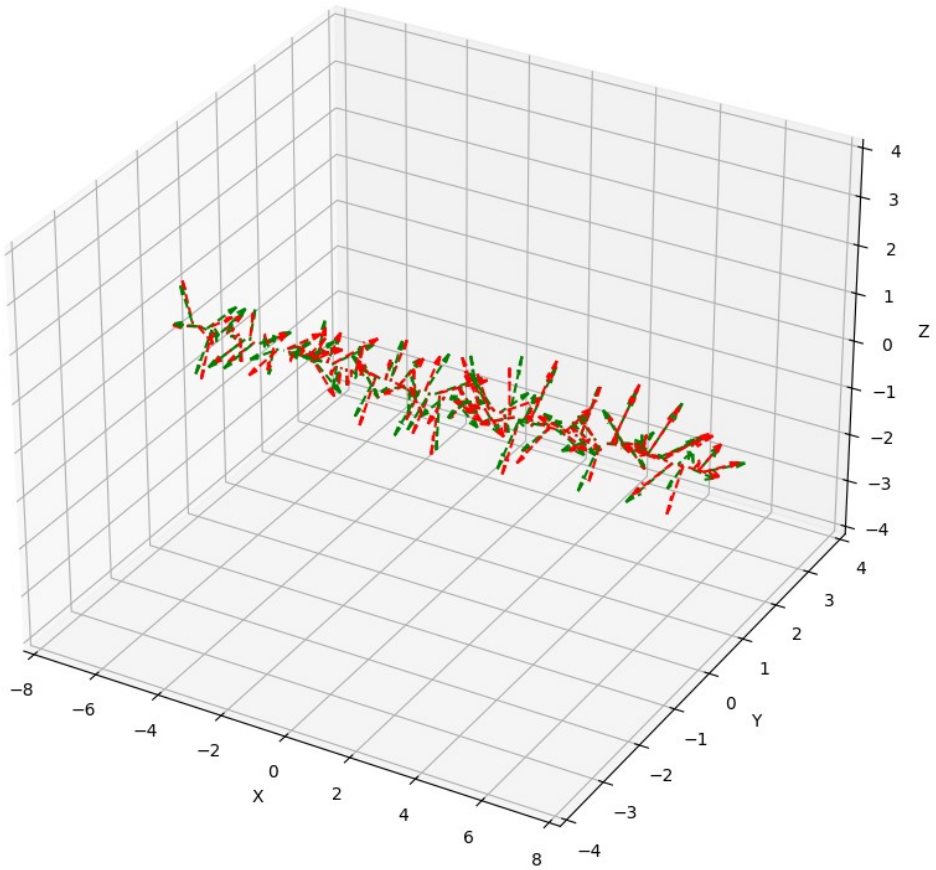

Figure 03

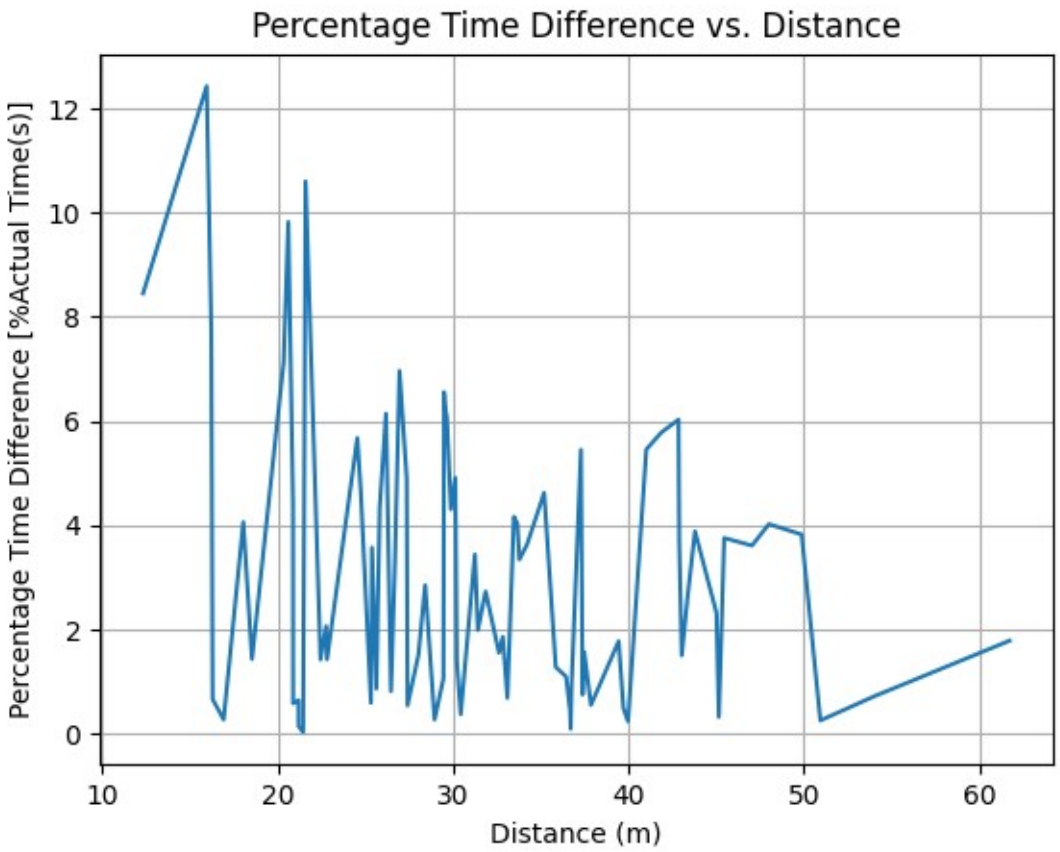

Figure 04

As mentioned before, two outputs together make model's estimations worse for at least one of them although there has been a lot of effort to put into this work to make better estimations.

5. Future Work

First of all the estimations, especially for the total time output can be more accurate. This can be achieved by further trials and errors with different settings for different arguments.

Secondly, this study can be combined with a path planning module and can be extended for whole trajectory generation. This could be possible after getting the total path to reach a goal, each segment of the path can be defined as new inputs for this DNN and trajectory variables can be given by the output of the model such as the control inputs that will result with the velocity vector that was given by the model.

6. Code Part

The summary for the code part of the project can be done such as following,

1. **Dataset Generation**:
    - The project starts by generating a dataset for trajectory optimization.
    - The dataset is loaded from files in a specified directory.

- The data consists of 15 features, including start position, start velocity, end position, end velocity, gate position and the last 4 columns include outputs of velocity vector, and time.

2. **Neural Network Model**:
    - The main neural network model (`mymodel`) is defined using PyTorch.
    - It consists of linear layers with ReLU activation functions.
    - The number of layers and units per layer can be adjusted.
    - The model predicts both position vectors and time.
3. **Optimization and Training**:
    - The `objective` function defines the optimization objective for Optuna.
    - The model is trained using a specified loss function (e.g., Mean Squared Error) and Adam optimizer.
    - The training process includes normalization of input data, batch processing, and evaluation of model performance.
    - Training and evaluation loops are implemented, with functions to train and evaluate the model.
4. **Evaluation and Visualization**:
    - The model's performance is evaluated by comparing its output to target variables.
    - Evaluation metrics include percentage error for time and position vectors.
    - Visualization functions are provided to plot loss curves, 3D vectors, and percentage time difference.
    - Evaluation results can be saved and plotted for analysis.
5. **Dataset Handling**:
    - Custom dataset classes are implemented using PyTorch's `Dataset` class.
    - Data loading functions are provided to load and preprocess datasets from files.
6. **Optimization and Final Model Selection**:
    - Optuna is used to search for the best hyperparameters and model architecture.
    - The best model is selected based on evaluation metrics and saved for future use.

References

[1] Liang Yang, Juntong Qi, J. Xiao, and Xia Yong, “A literature review of UAV 3D path planning,” in *Proceeding of the 11th World Congress on Intelligent Control and Automation*, Shenyang, China: IEEE, Jun. 2014, pp. 2376–2381. doi: 10.1109/WCICA.2014.7053093.

[2] F. Meyer and K. Glock, “Kinematic Orienteering Problem With Time-Optimal Trajectories for Multirotor UAVs,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 11402–11409, Oct. 2022, doi: 10.1109/LRA.2022.3194688.

[3] K. Teissing, “Replanning of Collision-Free Trajectories for Unmanned Aerial Vehicle”, May 2023.

[4] B. Li, G. Wu, Y. He, M. Fan, and W. Pedrycz, “An Overview and Experimental Study of Learning-based Optimization Algorithms for Vehicle Routing Problem.” arXiv, Feb. 19, 2022. Accessed: Feb. 15, 2024. [Online]. Available: http://arxiv.org/abs/2107.07076
[5] C. K. Joshi, T. Laurent, and X. Bresson, “An Efficient Graph Convolutional Network Technique for the Travelling Salesman Problem.” arXiv, Oct. 14, 2019. Accessed: Feb. 15, 2024. [Online]. Available: http://arxiv.org/abs/1906.01227
[6] W. Kool, H. van Hoof, and M. Welling, “Attention, Learn to Solve Routing Problems!” arXiv, Feb. 07, 2019. Accessed: Feb. 15, 2024. [Online]. Available: http://arxiv.org/abs/1803.08475
[7] L. Xin, W. Song, Z. Cao, and J. Zhang, “NeuroLKH: Combining Deep Learning Model with Lin-Kernighan-Helsgaun Heuristic for Solving the Traveling Salesman Problem.” arXiv, Oct. 15, 2021. Accessed: Feb. 15, 2024. [Online]. Available: http://arxiv.org/abs/2110.07983